

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-  
a162f3c86699fe76d.jsonl`  
- SHA-256: `e3d08d1bc4b6b6fd49d53edfca58bca12c0a5b56eda490e1c28d6e2eb81780e5`  
- Source modified: `2026-07-06T06:12:13+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:09:59.892Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact code at the cited lines, trace the real control flow, and if useful run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return REFUTED if the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff). Return PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension migration):

title: Migration 003 content_items backfill silently drops every video whose channel has no channels row (inner JOIN + INSERT OR IGNORE)
severity(claimed): blocker
file:line: src/tg_video_filter/db.py:562
summary: The content_items backfill inner-joins videos to sources on channel_id, but sources is

populated only from the channels table; any legacy video whose channel_id was never inserted into channels (routine for backfill-scanned channels) is silently discarded, so its content_item is never created.

failure_scenario: v1 backfill.py inserts videos via db.insert_video() but never calls db.upsert_channel(), and v1 videos.channel_id has NO foreign key to channels (git main:db.py CREATE TABLE videos, bare 'channel_id INTEGER NOT NULL'). So a production v1 DB routinely holds videos for channels absent from `channels`. On migration: sources is backfilled ONLY from channels (db.py:517-526), then content_items is backfilled with `FROM videos v JOIN sources s ON ... s.provider\_source\_id = CAST(v.channel\_id AS TEXT)` (db.py:562-567). The inner JOIN drops every video whose channel has no sources row; INSERT OR IGNORE masks it. Repro: seed users=111, channels only for -1001, videos for -1001 (msg 1, doc 111000) AND -2002 (msg 2, doc 222000); run init_db -> videos_deprecated=2 but content_items=1 (only msg 1). get_video_by_document_id(111,222000) returns None -> the video is invisible to the live schema; dedup breaks (it re-delivers) and after the one-release deprecated-table drop it is permanently lost.

evidence: src/tg_video_filter/db.py:562-567 (JOIN sources); db.py:517-526 (sources backfilled only from channels); db.py:539 (INSERT OR IGNORE); v1 `git show main:src/tg\_video\_filter/db.py` videos DDL has no FK to channels and insert_video does not upsert a channel; `git show main:backfill.py | grep -c upsert\_channel` == 0. Repro output: 'Legacy videos: 2 ... content_items rows (migrated): 1 ... DATA LOSS: 1 video(s) silently dropped by the JOIN ... get_video_by_document_id(222000) -> None'.
recommendation: Do not depend on the channels table for source existence. Before (or during) the content_items backfill, backfill sources for every DISTINCT (user_id, channel_id) found in videos (INSERT OR IGNORE INTO sources SELECT DISTINCT user_id,'telegram',CAST(channel_id AS TEXT),... FROM videos), or LEFT JOIN and create the missing source, so no video is dropped. Add a migration test seeding a video whose channel has no channels row and assert content_items count == videos count.

2. assistant (2026-07-06T06:10:03.942Z)

I'll verify this finding by examining the actual migration code and v1 behavior.

3. user (2026-07-06T06:10:05.439Z)

```
490         # net for one release per ADR 0010). The data-holding channels/videos/
491         # filter_config tables were already rebuilt above with FKs to accounts(id),
492         # so renaming users no longer retargets any live FK.
493         await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
494
495
496     @migration(3)
497     async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
498         """Land the provider-neutral domain tables (ADR 0011, Track B).
499
500         Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501         ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502         DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503         them), then backfills the three populated ones from the legacy
504         ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505         legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506         pattern). Idempotent: a no-op once the legacy tables have been renamed.
507         """
508         # Idempotency gate: if the legacy videos table is gone, this migration has
509         # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510         # dropped by _apply_schema post-003). Nothing to backfill.
511         if not await _table_exists(conn, "videos"):
512             return
513
514         # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515         #     channels stay watched (watched=1). provider_source_id is the Telegram
516         #     channel_id (cast to TEXT for the provider-neutral column).
517         await conn.execute(
518             """
519             INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
```

```

521         seen_first_at, last_seen_at)
522     SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523            1, seen_first_at, seen_first_at
524     FROM channels
525     """"
526 )
527
528 # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529 #   The three-branch decision tree (Option 1A):
530 #       a) document_id IS NOT NULL           ? 'tg:doc:' || document_id
531 #       b) all four fingerprint fields set   ? 'tg:fp:' || size:dur:h:w
532 #       c) else (Tier-0 fallback)           ? 'tg:msg:' || source_id:msg_id
533 #   The ``CASE`` is ordered strongest-first and each branch is guarded so
534 #   partial metadata never collapses to a degenerate key. source_id is
535 #   resolved by joining the channel's provider_source_id back to sources.id
536 #   (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """"
539     INSERT OR IGNORE INTO content_items
540         (account_id, source_id, provider, provider_message_id, content_type,
541          document_id, duration_s, width, height, size_bytes, mime_type,
542          dedup_key, matched, seen_at)
543     SELECT
544         v.user_id,
545         s.id,
546         'telegram',
547         CAST(v.message_id AS TEXT),
548         'video',
549         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551         CASE
552             WHEN v.document_id IS NOT NULL
553             THEN 'tg:doc:' || v.document_id
554             WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555                  AND v.height IS NOT NULL AND v.width IS NOT NULL
556             THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557                  || v.height || ':' || v.width
558             ELSE 'tg:msg:' || s.id || ':' || v.message_id
559         END,
560         v.matched,
561         v.seen_at
562     FROM videos v
563     JOIN sources s
564         ON s.account_id = v.user_id
565        AND s.provider = 'telegram'
566        AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """"
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 #   (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 #   this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """"
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576                                    created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """"
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 #   rollback safety net (ADR 0010 pattern).
584 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")

```

```

586         await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
587
588
589     async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590         """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591         async with conn.execute(
592             "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593         ) as cur:
594             return await cur.fetchone() is not None
595
596
597     # --- helpers -----
598
599
600     def _now() -> datetime:
601         """Timezone-aware UTC 'now', used for policy timestamps."""
602         return datetime.now(UTC)
603
604
605     def _row_to_account(row: aiosqlite.Row) -> Account:
606         return Account(
607             id=row["id"],
608             telegram_id=row["telegram_id"],
609             provider=row["provider"],

```

4. user (2026-07-06T06:10:06.961Z)

```

274:         CREATE TABLE videos (
275-             id            INTEGER PRIMARY KEY AUTOINCREMENT,
276-             user_id       INTEGER NOT NULL REFERENCES accounts(id),
277-             channel_id    INTEGER NOT NULL,
278-             message_id    INTEGER NOT NULL,
279-             document_id   INTEGER,
280-             duration_s    INTEGER,
281-             width         INTEGER,
282-             height        INTEGER,
283-             size_bytes    INTEGER,
284-             mime_type     TEXT,
285-             matched       INTEGER NOT NULL DEFAULT 0,
286-             forwarded     INTEGER NOT NULL DEFAULT 0,
287-             seen_at       TEXT NOT NULL,
288-             UNIQUE (user_id, channel_id, message_id)
289-         )
290-         """
291-         "id, user_id, channel_id, message_id, document_id, duration_s, width, "
292-         "height, size_bytes, mime_type, matched, forwarded, seen_at",
293-     ),
294-     (

```

5. assistant (2026-07-06T06:10:09.499Z)

Now let me verify the key claims: (1) v1 backfill inserts videos without upserting channels, (2) the JOIN behavior. Let me check the migration ordering and run a repro.

6. user (2026-07-06T06:10:15.617Z)

```

=== backfill.py upsert_channel count ===
0
=== insert_video in backfill ===
232:         inserted = await db.insert_video(conn, record)

```

7. assistant (2026-07-06T06:10:18.582Z)

The claim is that backfill inserts videos without upserting channels. But there are 0 matches

for `upsert\_channel`. Let me check what the actual channel-insertion function is named in v1, and whether backfill inserts channels by any other means.

8. user (2026-07-06T06:10:21.136Z)

```
=== channel functions in v1 db.py ===
383: def _row_to_channel(row: aiomysqlite.Row) -> Channel:
582: async def upsert_channel(conn: aiomysqlite.Connection, channel: Channel) -> Channel:
585:     """INSERT INTO channels (user_id, channel_id, title, seen_first_at)
670: async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
=== insert_video body in v1 ===
670: async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
671-     """Insert a new video row. Raises IntegrityError on a dedup-key collision.
672-
673-     Two dedup constraints protect against duplicates:
674-     - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
675-     - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
676-       same file forwarded across channels (content dedup Tier 1).
677-     """
678-     cur = await conn.execute(
679-         """INSERT INTO videos
680-             (user_id, channel_id, message_id, document_id, duration_s, width, height,
681-              size_bytes, mime_type, matched, forwarded, seen_at)
682-             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
683-         (
684-             video.user_id,
685-             video.channel_id,
686-             video.message_id,
687-             video.document_id,
688-             video.duration_s,
689-             video.width,
690-             video.height,
691-             video.size_bytes,
692-             video.mime_type,
693-             int(video.matched),
694-             int(video.forwarded),
695-             video.seen_at.isoformat(),
696-         ),
697-     )
698-     await conn.commit()
699-     return video.model_copy(update={"id": cur.lastrowid})
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-
801-
802-
803-
804-
805-
806-
807-
808-
809-
810-
811-
812-
813-
814-
815-
816-
817-
818-
819-
820-
821-
822-
823-
824-
825-
826-
827-
828-
829-
830-
831-
832-
833-
834-
835-
836-
837-
838-
839-
840-
841-
842-
843-
844-
845-
846-
847-
848-
849-
850-
851-
852-
853-
854-
855-
856-
857-
858-
859-
860-
861-
862-
863-
864-
865-
866-
867-
868-
869-
870-
871-
872-
873-
874-
875-
876-
877-
878-
879-
880-
881-
882-
883-
884-
885-
886-
887-
888-
889-
890-
891-
892-
893-
894-
895-
896-
897-
898-
899-
900-
901-
902-
903-
904-
905-
906-
907-
908-
909-
910-
911-
912-
913-
914-
915-
916-
917-
918-
919-
920-
921-
922-
923-
924-
925-
926-
927-
928-
929-
930-
931-
932-
933-
934-
935-
936-
937-
938-
939-
940-
941-
942-
943-
944-
945-
946-
947-
948-
949-
950-
951-
952-
953-
954-
955-
956-
957-
958-
959-
960-
961-
962-
963-
964-
965-
966-
967-
968-
969-
970-
971-
972-
973-
974-
975-
976-
977-
978-
979-
980-
981-
982-
983-
984-
985-
986-
987-
988-
989-
990-
991-
992-
993-
994-
995-
996-
997-
998-
999-
1000-
1001-
1002-
1003-
1004-
1005-
1006-
1007-
1008-
1009-
1010-
1011-
1012-
1013-
1014-
1015-
1016-
1017-
1018-
1019-
1020-
1021-
1022-
1023-
1024-
1025-
1026-
1027-
1028-
1029-
1030-
1031-
1032-
1033-
1034-
1035-
1036-
1037-
1038-
1039-
1040-
1041-
1042-
1043-
1044-
1045-
1046-
1047-
1048-
1049-
1050-
1051-
1052-
1053-
1054-
1055-
1056-
1057-
1058-
1059-
1060-
1061-
1062-
1063-
1064-
1065-
1066-
1067-
1068-
1069-
1070-
1071-
1072-
1073-
1074-
1075-
1076-
1077-
1078-
1079-
1080-
1081-
1082-
1083-
1084-
1085-
1086-
1087-
1088-
1089-
1090-
1091-
1092-
1093-
1094-
1095-
1096-
1097-
1098-
1099-
1100-
1101-
1102-
1103-
1104-
1105-
1106-
1107-
1108-
1109-
1110-
1111-
1112-
1113-
1114-
1115-
1116-
1117-
1118-
1119-
1120-
1121-
1122-
1123-
1124-
1125-
1126-
1127-
1128-
1129-
1130-
1131-
1132-
1133-
1134-
1135-
1136-
1137-
1138-
1139-
1140-
1141-
1142-
1143-
1144-
1145-
1146-
1147-
1148-
1149-
1150-
1151-
1152-
1153-
1154-
1155-
1156-
1157-
1158-
1159-
1160-
1161-
1162-
1163-
1164-
1165-
1166-
1167-
1168-
1169-
1170-
1171-
1172-
1173-
1174-
1175-
1176-
1177-
1178-
1179-
1180-
1181-
1182-
1183-
1184-
1185-
1186-
1187-
1188-
1189-
1190-
1191-
1192-
1193-
1194-
1195-
1196-
1197-
1198-
1199-
1200-
1201-
1202-
1203-
1204-
1205-
1206-
1207-
1208-
1209-
1210-
1211-
1212-
1213-
1214-
1215-
1216-
1217-
1218-
1219-
1220-
1221-
1222-
1223-
1224-
1225-
1226-
1227-
1228-
1229-
1230-
1231-
1232-
1233-
1234-
1235-
1236-
1237-
1238-
1239-
1240-
1241-
1242-
1243-
1244-
1245-
1246-
1247-
1248-
1249-
1250-
1251-
1252-
1253-
1254-
1255-
1256-
1257-
1258-
1259-
1260-
1261-
1262-
1263-
1264-
1265-
1266-
1267-
1268-
1269-
1270-
1271-
1272-
1273-
1274-
1275-
1276-
1277-
1278-
1279-
1280-
1281-
1282-
1283-
1284-
1285-
1286-
1287-
1288-
1289-
1290-
1291-
1292-
1293-
1294-
1295-
1296-
1297-
1298-
1299-
1300-
1301-
1302-
1303-
1304-
1305-
1306-
1307-
1308-
1309-
1310-
1311-
1312-
1313-
1314-
1315-
1316-
1317-
1318-
1319-
1320-
1321-
1322-
1323-
1324-
1325-
1326-
1327-
1328-
1329-
1330-
1331-
1332-
1333-
1334-
1335-
1336-
1337-
1338-
1339-
1340-
1341-
1342-
1343-
1344-
1345-
1346-
1347-
1348-
1349-
1350-
1351-
1352-
1353-
1354-
1355-
1356-
1357-
1358-
1359-
1360-
1361-
1362-
1363-
1364-
1365-
1366-
1367-
1368-
1369-
1370-
1371-
1372-
1373-
1374-
1375-
1376-
1377-
1378-
1379-
1380-
1381-
1382-
1383-
1384-
1385-
1386-
1387-
1388-
1389-
1390-
1391-
1392-
1393-
1394-
1395-
1396-
1397-
1398-
1399-
1400-
1401-
1402-
1403-
1404-
1405-
1406-
1407-
1408-
1409-
1410-
1411-
1412-
1413-
1414-
1415-
1416-
1417-
1418-
1419-
1420-
1421-
1422-
1423-
1424-
1425-
1426-
1427-
1428-
1429-
1430-
1431-
1432-
1433-
1434-
1435-
1436-
1437-
1438-
1439-
1440-
1441-
1442-
1443-
1444-
1445-
1446-
1447-
1448-
1449-
1450-
1451-
1452-
1453-
1454-
1455-
1456-
1457-
1458-
1459-
1460-
1461-
1462-
1463-
1464-
1465-
1466-
1467-
1468-
1469-
1470-
1471-
1472-
1473-
1474-
1475-
1476-
1477-
1478-
1479-
1480-
1481-
1482-
1483-
1484-
1485-
1486-
1487-
1488-
1489-
1490-
1491-
1492-
1493-
1494-
1495-
1496-
1497-
1498-
1499-
1500-
1501-
1502-
1503-
1504-
1505-
1506-
1507-
1508-
1509-
1510-
1511-
1512-
1513-
1514-
1515-
1516-
1517-
1518-
1519-
1520-
1521-
1522-
1523-
1524-
1525-
1526-
1527-
1528-
1529-
1530-
1531-
1532-
1533-
1534-
1535-
1536-
1537-
1538-
1539-
1540-
1541-
1542-
1543-
1544-
1545-
1546-
1547-
1548-
1549-
1550-
1551-
1552-
1553-
1554-
1555-
1556-
1557-
1558-
1559-
1560-
1561-
1562-
1563-
1564-
1565-
1566-
1567-
1568-
1569-
1570-
1571-
1572-
1573-
1574-
1575-
1576-
1577-
1578-
1579-
1580-
1581-
1582-
1583-
1584-
1585-
1586-
1587-
1588-
1589-
1590-
1591-
1592-
1593-
1594-
1595-
1596-
1597-
1598-
1599-
1600-
1601-
1602-
1603-
1604-
1605-
1606-
1607-
1608-
1609-
1610-
1611-
1612-
1613-
1614-
1615-
1616-
1617-
1618-
1619-
1620-
1621-
1622-
1623-
1624-
1625-
1626-
1627-
1628-
1629-
1630-
1631-
1632-
1633-
1634-
1635-
1636-
1637-
1638-
1639-
1640-
1641-
1642-
1643-
1644-
1645-
1646-
1647-
1648-
1649-
1650-
1651-
1652-
1653-
1654-
1655-
1656-
1657-
1658-
1659-
1660-
1661-
1662-
1663-
1664-
1665-
1666-
1667-
1668-
1669-
1670-
1671-
1672-
1673-
1674-
1675-
1676-
1677-
1678-
1679-
1680-
1681-
1682-
1683-
1684-
1685-
1686-
1687-
1688-
1689-
1690-
1691-
1692-
1693-
1694-
1695-
1696-
1697-
1698-
1699-
1700-
1701-
1702-
1703-
1704-
1705-
1706-
1707-
1708-
1709-
1710-
1711-
1712-
1713-
1714-
1715-
1716-
1717-
1718-
1719-
1720-
1721-
1722-
1723-
1724-
1725-
1726-
1727-
1728-
1729-
1730-
1731-
1732-
1733-
1734-
1735-
1736-
1737-
1738-
1739-
1740-
1741-
1742-
1743-
1744-
1745-
1746-
1747-
1748-
1749-
1750-
1751-
1752-
1753-
1754-
1755-
1756-
1757-
1758-
1759-
1760-
1761-
1762-
1763-
1764-
1765-
1766-
1767-
1768-
1769-
1770-
1771-
1772-
1773-
1774-
1775-
1776-
1777-
1778-
1779-
1780-
1781-
1782-
1783-
1784-
1785-
1786-
1787-
1788-
1789-
1790-
1791-
1792-
1793-
1794-
1795-
1796-
1797-
1798-
1799-
1800-
1801-
1802-
1803-
1804-
1805-
1806-
1807-
1808-
1809-
1810-
1811-
1812-
1813-
1814-
1815-
1816-
1817-
1818-
1819-
1820-
1821-
1822-
1823-
1824-
1825-
1826-
1827-
1828-
1829-
1830-
1831-
1832-
1833-
1834-
1835-
1836-
1837-
1838-
1839-
1840-
1841-
1842-
1843-
1844-
1845-
1846-
1847-
1848-
1849-
1850-
1851-
1852-
1853-
1854-
1855-
1856-
1857-
1858-
1859-
1860-
1861-
1862-
1863-
1864-
1865-
1866-
1867-
1868-
1869-
1870-
1871-
1872-
1873-
1874-
1875-
1876-
1877-
1878-
1879-
1880-
1881-
1882-
1883-
1884-
1885-
1886-
1887-
1888-
1889-
1890-
1891-
1892-
1893-
1894-
1895-
1896-
1897-
1898-
1899-
1900-
1901-
1902-
1903-
1904-
1905-
1906-
1907-
1908-
1909-
1910-
1911-
1912-
1913-
1914-
1915-
1916-
1917-
1918-
1919-
1920-
1921-
1922-
1923-
1924-
1925-
1926-
1927-
1928-
1929-
1930-
1931-
1932-
1933-
1934-
1935-
1936-
1937-
1938-
1939-
1940-
1941-
1942-
1943-
1944-
1945-
1946-
1947-
1948-
1949-
1950-
1951-
1952-
1953-
1954-
1955-
1956-
1957-
1958-
1959-
1960-
1961-
1962-
1963-
1964-
1965-
1966-
1967-
1968-
1969-
1970-
1971-
1972-
1973-
1974-
1975-
1976-
1977-
1978-
1979-
1980-
1981-
1982-
1983-
1984-
1985-
1986-
1987-
1988-
1989-
1990-
1991-
1992-
1993-
1994-
1995-
1996-
1997-
1998-
1999-
2000-
2001-
2002-
2003-
2004-
2005-
2006-
2007-
2008-
2009-
2010-
2011-
2012-
2013-
2014-
2015-
2016-
2017-
2018-
2019-
2020-
2021-
2022-
2023-
2024-
2025-
2026-
2027-
2028-
2029-
2030-
2031-
2032-
2033-
2034-
2035-
2036-
2037-
2038-
2039-
2040-
2041-
2042-
2043-
2044-
2045-
2046-
2047-
2048-
2049-
2050-
2051-
2052-
2053-
2054-
2055-
2056-
2057-
2058-
2059-
2060-
2061-
2062-
2063-
2064-
2065-
2066-
2067-
2068-
2069-
2070-
2071-
2072-
2073-
2074-
2075-
2076-
2077-
2078-
2079-
2080-
2081-
2082-
2083-
2084-
2085-
2086-
2087-
2088-
2089-
2090-
2091-
2092-
2093-
2094-
2095-
2096-
2097-
2098-
2099-
2100-
2101-
2102-
2103-
2104-
2105-
2106-
2107-
2108-
2109-
2110-
2111-
2112-
2113-
2114-
2115-
2116-
2117-
2118-
2119-
2120-
2121-
2122-
2123-
2124-
2125-
2126-
2127-
2128-
2129-
2130-
2131-
2132-
2133-
2134-
2135-
2136-
2137-
2138-
2139-
2140-
2141-
2142-
2143-
2144-
2145-
2146-
2147-
2148-
2149-
2150-
2151-
2152-
2153-
2154-
2155-
2156-
2157-
2158-
2159-
2160-
2161-
2162-
2163-
2164-
2165-
2166-
2167-
2168-
2169-
2170-
2171-
2172-
2173-
2174-
2175-
2176-
2177-
2178-
2179-
2180-
2181-
2182-
2183-
2184-
2185-
2186-
2187-
2188-
2189-
2190-
2191-
2192-
2193-
2194-
2195-
2196-
2197-
2198-
2199-
2200-
2201-
2202-
2203-
2204-
2205-
2206-
2207-
2208-
2209-
2210-
2211-
2212-
2213-
2214-
2215-
2216-
2217-
2218-
2219-
2220-
2221-
2222-
2223-
2224-
2225-
2226-
2227-
2228-
2229-
2230-
2231-
2232-
2233-
2234-
2235-
2236-
2237-
2238-
2239-
2240-
2241-
2242-
2243-
2244-
2245-
2246-
2247-
2248-
2249-
2250-
2251-
2252-
2253-
2254-
2255-
2256-
2257-
2258-
2259-
2260-
2261-
2262-
2263-
2264-
2265-
2266-
2267-
2268-
2269-
2270-
2271-
2272-
2273-
2274-
2275-
2276-
2277-
2278-
2279-
2280-
2281-
2282-
2283-
2284-
2285-
2286-
2287-
2288-
2289-
2290-
2291-
2292-
2293-
2294-
2295-
2296-
2297-
2298-
2299-
2300-
2301-
2302-
2303-
2304-
2305-
2306-
2307-
2308-
2309-
2310-
2311-
2312-
2313-
2314-
2315-
2316-
2317-
2318-
2319-
2320-
2321-
2322-
2323-
2324-
2325-
2326-
2327-
2328-
2329-
2330-
2331-
2332-
2333-
2334-
2335-
2336-
2337-
2338-
2339-
2340-
2341-
2342-
2343-
2344-
2345-
2346-
2347-
2348-
2349-
2350-
2351-
2352-
2353-
2354-
2355-
2356-
2357-
2358-
2359-
2360-
2361-
2362-
2363-
2364-
2365-
2366-
2367-
2368-
2369-
2370-
2371-
2372-
2373-
2374-
2375-
2376-
2377-
2378-
2379-
2380-
2381-
2382-
2383-
2384-
2385-
2386-
2387-
2388-
2389-
2390-
2391-
2392-
2393-
2394-
2395-
2396-
2397-
2398-
2399-
2400-
2401-
2402-
2403-
2404-
2405-
2406-
2407-
2408-
2409-
2410-
2411-
2412-
2413-
2414-
2415-
2416-
2417-
2418-
2419-
2420-
2421-
2422-
2423-
2424-
2425-
2426-
2427-
2428-
2429-
2430-
2431-
2432-
2433-
2434-
2435-
2436-
2437-
2438-
2439-
2440-
2441-
2442-
2443-
2444-
2445-
2446-
2447-
2448-
2449-
2450-
2451-
2452-
2453-
2454-
2455-
2456-
2457-
2458-
2459-
2460-
2461-
2462-
2463-
2464-
2465-
2466-
2467-
2468-
2469-
2470-
2471-
2472-
2473-
2474-
2475-
2476-
2477-
2478-
2479-
2480-
2481-
2482-
2483-
2484-
2485-
2486-
2487-
2488-
2489-
2490-
2491-
2492-
2493-
2494-
2495-
2496-
2497-
2498-
2499-
2500-
2501-
2502-
2503-
2504-
2505-
2506-
2507-
2508-
2509-
2510-
2511-
2512-
2513-
2514-
2515-
2516-
2517-
2518-
2519-
2520-
2521-
2522-
2523-
2524-
2525-
2526-
2527-
2528-
2529-
2530-
2531-
2532-
2533-
2534-
2535-
2536-
2537-
2538-
2539-
2540-
2541-
2542-
2543-
2544-
2545-
2546-
2547-
2548-
2549-
2550-
2551-
2552-
2553-
2554-
2555-
2556-
2557-
2558-
2559-
2560-
2561-
2562-
2563-
2564-
2565-
2566-
2567-
2568-
2569-
2570-
2571-
2572-
2573-
2574-
2575-
2576-
2577-
2578-
2579-
2580-
2581-
2582-
2583-
2584-
2585-
2586-
2587-
2588-
2589-
2590-
2591-
2592-
2593-
2594-
2595-
2596-
2597-
2598-
2599-
2600-
2601-
2602-
2603-
2604-
2605-
2606-
2607-
2608-
2609-
2610-
2611-
2612-
2613-
2614-
2615-
2616-
2617-
2618-
2619-
2620-
2621-
2622-
2623-
2624-
2625-
2626-
2627-
2628-
2629-
2630-
2631-
2632-
2633-
2634-
2635-
2636-
2637-
2638-
2639-
2640-
2641-
2642-
2643-
2644-
2645-
2646-
2647-
2648-
2649-
2650-
2651-
2652-
2653-
2654-
2655-
2656-
2657-
2658-
2659-
2660-
2661-
2662-
2663-
2664-
2665-
2666-
2667-
2668-
2669-
2670-
2671-
2672-
2673-
2674-
2675-
2676-
2677-
2678-
2679-
2680-
2681-
2682-
2683-
2684-
2685-
2686-
2687-
2688-
2689-
2690-
2691-
2692-
2693-
2694-
2695-
2696-
2697-
2698-
2699-
```

```

        if existing is not None:
            log.info(
                "backfill_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
                "channel=%s msg=%s original_channel=%s original_msg=%s",
                user.id,
                meta.size_bytes,
                meta.duration_s,
                meta.height,
                meta.width,
                channel_id,
                message_id,
                existing.channel_id,
                existing.message_id,
            )
            continue

        summary["videos_scanned"] += 1
        dialog_videos += 1

        result = engine.evaluate(meta)
        record = VideoRecord(
            user_id=user.id,
            channel_id=channel_id,
            message_id=message_id,
            document_id=document_id, # type: ignore[arg-type]
            duration_s=meta.duration_s,
            width=meta.width,
            height=meta.height,
            size_bytes=meta.size_bytes,
            mime_type=meta.mime_type,
            matched=result.matched,
        )
        inserted = await db.insert_video(conn, record)

        if result.matched:
            summary["videos_matched"] += 1
            dialog_matched += 1
            link_text = build_link_text(
                message_link=getattr(message, "link", None),
                duration_s=meta.duration_s,
                height=meta.height,
                size_bytes=meta.size_bytes,
                channel_title=title, # from the dialog object
            )
            await deliver(
                client,

```

11. assistant (2026-07-06T06:10:31.966Z)

Confirmed: v1 backfill inserts videos for scanned channels via `insert\_video` but never calls `upsert\_channel`. So a production v1 DB routinely holds videos whose channel_id is absent from the channels table. Now let me run the repro to confirm the data loss.

12. assistant (2026-07-06T06:10:47.847Z)

```

import asyncio, aiosqlite
from src.tg_video_filter import db

```

```

async def main():
    conn = await aiosqlite.connect(":memory:")
    conn.row_factory = aiosqlite.Row
    await conn.execute("PRAGMA foreign_keys=ON")

```

```

# Build a PRE-ADR-0011 legacy schema: accounts + channels + videos + filter_config.
# We simulate by applying v1 legacy tables manually (migration 003 expects them).

```

```

await conn.executescript("""
CREATE TABLE accounts (id INTEGER PRIMARY KEY AUTOINCREMENT, telegram_id INTEGER, provider
TEXT, created_at TEXT);
CREATE TABLE users_deprecated (id INTEGER PRIMARY KEY, telegram_id INTEGER);
CREATE TABLE channels (
    id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL REFERENCES accounts(id),
    channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL, UNIQUE(user_id,
channel_id));
CREATE TABLE videos (
    id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL REFERENCES accounts(id),
    channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, document_id INTEGER,
    duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER, mime_type TEXT,
    matched INTEGER NOT NULL DEFAULT 0, forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT
NULL,
    UNIQUE(user_id, channel_id, message_id));
CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY REFERENCES accounts(id), config_json
TEXT NOT NULL);
""")
await conn.execute("INSERT INTO accounts (id, telegram_id, provider, created_at) VALUES
(111,111,'telegram','2020-01-01')")
# channels row ONLY for -1001
await conn.execute("INSERT INTO channels (user_id, channel_id, title, seen_first_at) VALUES
(111,-1001,'Chan A','2020-01-01')")
# video for -1001 (has channels row) AND -2002 (NO channels row -- backfill-scanned)
await conn.execute("INSERT INTO videos (user_id, channel_id, message_id, document_id,
seen_at) VALUES (111,-1001,1,111000,'2020-01-02')")
await conn.execute("INSERT INTO videos (user_id, channel_id, message_id, document_id,
seen_at) VALUES (111,-2002,2,222000,'2020-01-03')")
await conn.execute("INSERT INTO filter_config (user_id, config_json) VALUES (111,'{}')")
await conn.commit()

# set user_version to 2 so migration 003 runs
await conn.execute("PRAGMA user_version = 2")
await conn.commit()

await db._apply_schema(conn)

async with conn.execute("SELECT COUNT(*) c FROM videos_deprecated") as cur:
    vd = (await cur.fetchone())["c"]
async with conn.execute("SELECT COUNT(*) c FROM content_items") as cur:
    ci = (await cur.fetchone())["c"]
async with conn.execute("SELECT COUNT(*) c FROM sources") as cur:
    src = (await cur.fetchone())["c"]
print(f"Legacy videos (videos_deprecated): {vd}")
print(f"sources rows: {src}")
print(f"content_items rows (migrated): {ci}")
if ci < vd:
    print(f"DATA LOSS: {vd-ci} video(s) silently dropped by the JOIN")
# try get_video_by_document_id for the dropped one
async with conn.execute("SELECT id FROM content_items WHERE document_id='222000'") as cur:
    row = await cur.fetchone()
print(f"content_item for document_id 222000 -> {row}")
await conn.close()

asyncio.run(main())

```

13. user (2026-07-06T06:10:47.848Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

14. assistant (2026-07-06T06:10:51.049Z)

Let me check `\_apply\_schema` and how migrations are invoked before running a repro, so the harness is correct.

15. user (2026-07-06T06:10:53.211Z)

```
185:     then runs any pending migrations via the ``user_version`` ladder.
188:     await _apply_schema(conn)
191:async def _apply_schema(conn: aiosqlite.Connection) -> None:
214:# Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
220:# 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
221:# 2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
225:_MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
228:def _migration(
234:     """Register a migration that brings the DB up to ``user_version == version``."""
239:     _MIGRATIONS.append((version, fn))
308:     """Run every registered migration whose version > current ``user_version``."""
309:     async with conn.execute("PRAGMA user_version") as cur:
313:         for version, fn in _MIGRATIONS:
317:             await conn.execute(f"PRAGMA user_version = {version}")
327:@migration(1)
328:async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
344:@migration(2)
345:async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
496:@migration(3)
497:async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
510:     # dropped by _apply_schema post-003). Nothing to backfill.
```

16. user (2026-07-06T06:10:58.090Z)

```
191     async def _apply_schema(conn: aiosqlite.Connection) -> None:
192         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193         # and a busy timeout so concurrent access waits instead of failing immediately.
194         await conn.execute("PRAGMA journal_mode = WAL")
195         await conn.execute("PRAGMA foreign_keys = ON")
196         await conn.execute("PRAGMA busy_timeout = 5000")
197         await conn.executescript(_SCHEMA)
198         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
201         # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202         # any *empty* recreations of those legacy tables from a subsequent
203         # ``init_db`` are dropped here so they don't shadow the renamed copies.
204         if await _table_exists(conn, "channels_deprecated"):
205             await conn.execute("DROP TABLE IF EXISTS channels")
206             await conn.execute("DROP TABLE IF EXISTS videos")
207             await conn.execute("DROP TABLE IF EXISTS filter_config")
208         await _apply_migrations(conn)
209         await conn.commit()
210
211
212     # --- migrations -----
213     #
214     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
215     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216     # the source of truth for *fresh* databases; the migrations below reconcile
217     # *existing* databases that were created by an older schema version.
218     #
219     # To add a new migration:
220     # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
```

17. assistant (2026-07-06T06:11:04.272Z)

Now I understand the flow. `\_apply\_schema` runs `\_SCHEMA` (which creates legacy tables + new domain tables), then runs migrations. For a legacy DB at user_version=2, migration 003 runs the backfill. Let me write a cleaner repro that mimics a real legacy DB by applying the v1 schema at version 2, then calling `\_apply\_schema`.

Rather than hand-build, I'll checkout v1's `\_apply\_schema` path. But simpler: seed the legacy

tables, set user_version=2, then run migration 003 directly. Let me write the repro.

18. user (2026-07-06T06:11:27.812Z)

Exit code 1

Traceback (most recent call last):

```
File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 2, in <module>
```

```
    from src.tg_video_filter import db
```

ModuleNotFoundError: No module named 'src'

19. user (2026-07-06T06:11:37.442Z)

```
src/tg_video_filter/__init__.py
```

```
x
```

```
Legacy videos (videos_deprecated): 2
```

```
sources rows: 1
```

```
content_items rows (migrated): 1
```

```
*** DATA LOSS: 1 video(s) silently dropped by the JOIN ***
```

```
content_item for document_id 222000 (the orphan-channel video) -> None
```

20. assistant (2026-07-06T06:11:41.404Z)

The repro confirms it. But I must verify one thing for the adversarial check: is it realistic that a v1 production DB has videos whose channel is absent from `channels`? I confirmed backfill.py never calls `upsert\_channel`. Let me also verify the live-watch path (the non-backfill ingestion) DOES upsert channels? because if EVERY video path upserts channels, then orphan videos couldn't exist. The finding hinges on backfill specifically. Let me check callers of `insert\_video` and `upsert\_channel`.

21. user (2026-07-06T06:11:43.828Z)

```
=== callers of upsert_channel on main ===
```

```
103:     """Scan the last ``days`` of videos in all of ``user``'s channels.
```

```
344:         await _send_progress(f"? Starting backfill of all channels (last  
{duration_label})...")
```

```
391:         f"? Backfill complete: {summary['dialog_count']} channels scanned, "
```

```
--- grep whole main tree ---
```

```
main:src/tg_video_filter/db.py:582:async def upsert_channel(conn: aiosqlite.Connection, channel:  
Channel) -> Channel:
```

```
main:src/tg_video_filter/db.py:763:     "upsert_channel",
```

```
main:src/tg_video_filter/telethon_client.py:240:         await db.upsert_channel(
```

22. assistant (2026-07-06T06:11:52.793Z)

This is decisive. On main, `upsert\_channel` is called ONLY from `telethon\_client.py:240` (the live-watch path). The backfill path (`backfill.py`) inserts videos via `insert\_video` at line 232 but NEVER calls `upsert\_channel`. So videos scanned by backfill for channels the user hasn't live-watched will have NO corresponding `channels` row. This is a routine, realistic v1 production state.

The migration 003 content_items backfill inner-joins `videos` to `sources` (which is populated only from `channels`), so every such backfill-only video is silently dropped by the inner JOIN, masked by `INSERT OR IGNORE`. Repro confirms: 2 legacy videos, 1 content_item, orphan-channel video's content_item is None.

Let me do one final check: confirm `sources` is truly populated ONLY from `channels` in this migration (no other backfill source), and that there's no compensating LEFT JOIN or fallback.

23. user (2026-07-06T06:11:54.129Z)

```
519:         INSERT OR IGNORE INTO sources
```

```
905:         """INSERT INTO sources
```

24. assistant (2026-07-06T06:12:04.409Z)

Line 519 is the migration backfill (from `channels` only). Line 905 is the runtime `upsert\_source` helper, not part of the migration. So within migration 003, `sources` is backfilled exclusively from `channels`, and the `content\_items` backfill inner-joins to `sources`. No LEFT JOIN, no fallback. The finding holds.

All claims verified:

- v1 `videos.channel\_id` has no FK to `channels` (main db.py:277, bare `channel\_id INTEGER NOT NULL`).
- v1 backfill inserts videos (backfill.py:232) but never upserts channels; `upsert\_channel` is called only from the live-watch path (telethon_client.py:240). So orphan-channel videos are routine.
- Migration 003 backfills `sources` only from `channels` (db.py:517-526), then inner-joins `videos` to `sources` for `content\_items` (db.py:562-567) with `INSERT OR IGNORE` (db.py:539).
- Repro: 2 legacy videos ? 1 content_item; orphan-channel video (document_id 222000) has no content_item.

Consequence: after the one-release deprecated-table drop, these videos are permanently lost from the live schema, and dedup breaks (re-delivery) in the interim. This is silent data loss / wrong core behaviour = blocker.

25. user (2026-07-06T06:12:13.532Z)

Structured output provided successfully